INSTITUTO POLITÉCNICO NACIONAL
UNIDAD PROFESIONAL
INTERDISCIPLINARIA DE INGENIERÍA Y
CIENCIAS SOCIALES Y ADMINISTRATIVAS

Academia de Informática

6CM68

Proyecto:

Sistema de Gestión Bibliotecaria

Documentación técnica del proyecto

Integrantes:

Centeno Medina Ángel Rafael
Estrada Robles José Roberto
García García Julio César
Jiménez Castelan Jesús Antonio

Materia:

Bases de Datos No SQL

Profesor:

Yventz Entzana Garduño

Ciudad de México, noviembre de 2026

Índice

1 Definición del proyecto y alcance **1**

1.1 Justificación del uso de una base de datos NoSQL 1

1.2 Alcance 1

2 Objetivos del sistema **2**

3 Arquitectura del sistema **2**

3.1 Capas de la arquitectura 2

3.2 Flujo de un préstamo 3

4 Tecnologías utilizadas **5**

5 Origen de datos (MongoDB) **5**

5.1 Colecciones y campos 6

5.2 Relaciones entre colecciones 7

5.3 Ejemplos de documentos 7

5.4 Justificación del modelado NoSQL 8

5.5 Índices 9

5.6 Datos de prueba 9

6 Implementación del backend **10**

6.1 Estructura del proyecto 10

6.2 API REST 11

6.3 Reglas de negocio 11

6.4 Reporte de libros más solicitados 12

6.5 Manejo de errores 13

7 Interfaz web **13**

8 Instalación y ejecución **15**

8.1 Requisitos previos 15

8.2 Pasos 15
8.3 Problemas frecuentes 16

9 Pruebas **16**
9.1 Pruebas funcionales 16
9.2 Plan de pruebas de la API 16

10 Gestión del proyecto **18**
10.1 Planeación en Jira 18
10.2 Control de versiones 18

11 Estado de cumplimiento, limitaciones y trabajo futuro **18**
11.1 Cumplimiento de los objetivos 18
11.2 Limitaciones 19
11.3 Trabajo futuro 19

12 Conclusiones **20**

A Script del modelo de datos (mongosh) **20**

Índice de figuras

1 Arquitectura del sistema en capas. La línea punteada indica un componente previsto que aún no se implementa. . . . 3
2 Validaciones del endpoint de préstamos y respuesta en cada caso. . . . 4
3 Modelo de datos: `prestamos` referencia a `usuarios` y `libros` por `ObjectId`; los autores van embebidos en `libros`. . . . 7
4 Módulo de catálogo filtrado por la categoría Terror (55 libros). . . . 14
5 Búsqueda de un usuario por boleta al registrar un préstamo. . . . 14
6 Módulo de usuarios. . . . 15

Índice de cuadros

1 Stack tecnológico del proyecto. . . . 5
2 Estructura de la colección `libros`. . . . 6
3 Estructura de la colección `usuarios`. . . . 6
4 Estructura de la colección `prestamos`. . . . 7
5 Índices definidos para optimizar los tiempos de respuesta. . . . 9
6 Conjuntos de datos de prueba. . . . 9
7 Endpoints de la API REST. . . . 11
8 Códigos de error devueltos por la API. . . . 13
9 Problemas frecuentes durante la instalación. . . . 16
10 Plan de pruebas de la API REST. . . . 17
11 Épicas del proyecto. . . . 18
12 Grado de cumplimiento de los objetivos específicos. . . . 18

1 Definición del proyecto y alcance

El proyecto consiste en el desarrollo de una aplicación web orientada a la administración integral de una biblioteca universitaria mediante el uso de una base de datos NoSQL. El sistema permite gestionar de forma eficiente el registro de libros y autores, la información de estudiantes y profesores, así como el control riguroso de préstamos y devoluciones.

1.1 Justificación del uso de una base de datos NoSQL

Una biblioteca maneja información con estructuras variables: un libro puede tener uno o varios autores, distintas ediciones y atributos que cambian según el tipo de material. MongoDB, al almacenar documentos JSON (BSON), permite representar cada libro como un único documento autocontenido, con sus autores embebidos como arreglo, evitando las uniones entre tablas que exigiría un modelo relacional. Además, su modelo flexible facilita incorporar nuevos campos (por ejemplo, formato digital o idioma) sin migraciones costosas, y sus índices de texto y compuestos aceleran las búsquedas bibliográficas y la detección de préstamos vencidos.

1.2 Alcance

El sistema incluye

- Catálogo de libros con búsqueda por título, autor, categoría o ISBN, filtros y paginación.
- Registro de usuarios (estudiantes y profesores) y búsqueda por nombre, boleta o correo.
- Préstamos y devoluciones con control de inventario de ejemplares.
- Detección de préstamos vencidos y bloqueo de nuevos préstamos a usuarios con retrasos.
- Reporte de libros más solicitados.
- API REST y base de datos documental con validación de esquemas e índices.

Fuera del alcance de esta versión

- Autenticación y roles de acceso al sistema (administrador, bibliotecario).
- Envío de notificaciones externas (correo o mensajería) por préstamos vencidos.
- Multas, reservaciones y renovaciones de préstamos.

2 Objetivos del sistema

Objetivo general

Diseñar e implementar un sistema de gestión bibliotecaria utilizando una base de datos NoSQL, con el fin de administrar de manera flexible y eficiente los libros, usuarios, préstamos y devoluciones.

Objetivos específicos

O1. Registrar, consultar, actualizar y eliminar libros del catálogo académico.

O2. Administrar la información de estudiantes y profesores en el módulo de usuarios.

O3. Registrar y controlar de forma transaccional los préstamos y devoluciones.

O4. Consultar la disponibilidad en tiempo real de los libros.

O5. Generar alertas automatizadas sobre préstamos vencidos.

O6. Implementar operaciones CRUD completas en MongoDB mediante una interfaz optimizada.

O7. Crear motores de búsqueda eficientes por título, autor, categoría o ISBN.

O8. Generar reportes analíticos de libros más solicitados y usuarios con retrasos.

O9. Aplicar índices de base de datos para optimizar los tiempos de respuesta en las consultas.

O10. Diseñar una API REST robusta para conectar la persistencia de datos con la interfaz web.

El grado de avance de cada objetivo se resume en la sección 11.

3 Arquitectura del sistema

3.1 Capas de la arquitectura

La arquitectura se divide en capas claramente definidas, de modo que cada una pueda evolucionar sin afectar a las demás (figura 1):

Capa de presentación (frontend).
Interfaz web desarrollada con HTML, CSS y JavaScript sin frameworks. Consume la API mediante `fetch` y presenta tres módulos: catálogo, préstamos y usuarios.

Capa de lógica (backend API REST).
Servidor sobre Node.js y Express.js, encargado de procesar solicitudes HTTP/JSON, aplicar las reglas de negocio (disponibilidad, cuentas suspendidas, vencidos) y ejecutar las operaciones CRUD.

Capa de persistencia (origen de datos NoSQL).
MongoDB, modelado y validado desde la aplicación mediante Mongoose (ODM) y desde la propia base de datos con validadores `$jsonSchema`.

Módulo de alertas.
Componente de avisos sobre préstamos vencidos. En esta versión, el backend detecta y marca automáticamente los préstamos vencidos; el envío de notificaciones externas queda como trabajo futuro.

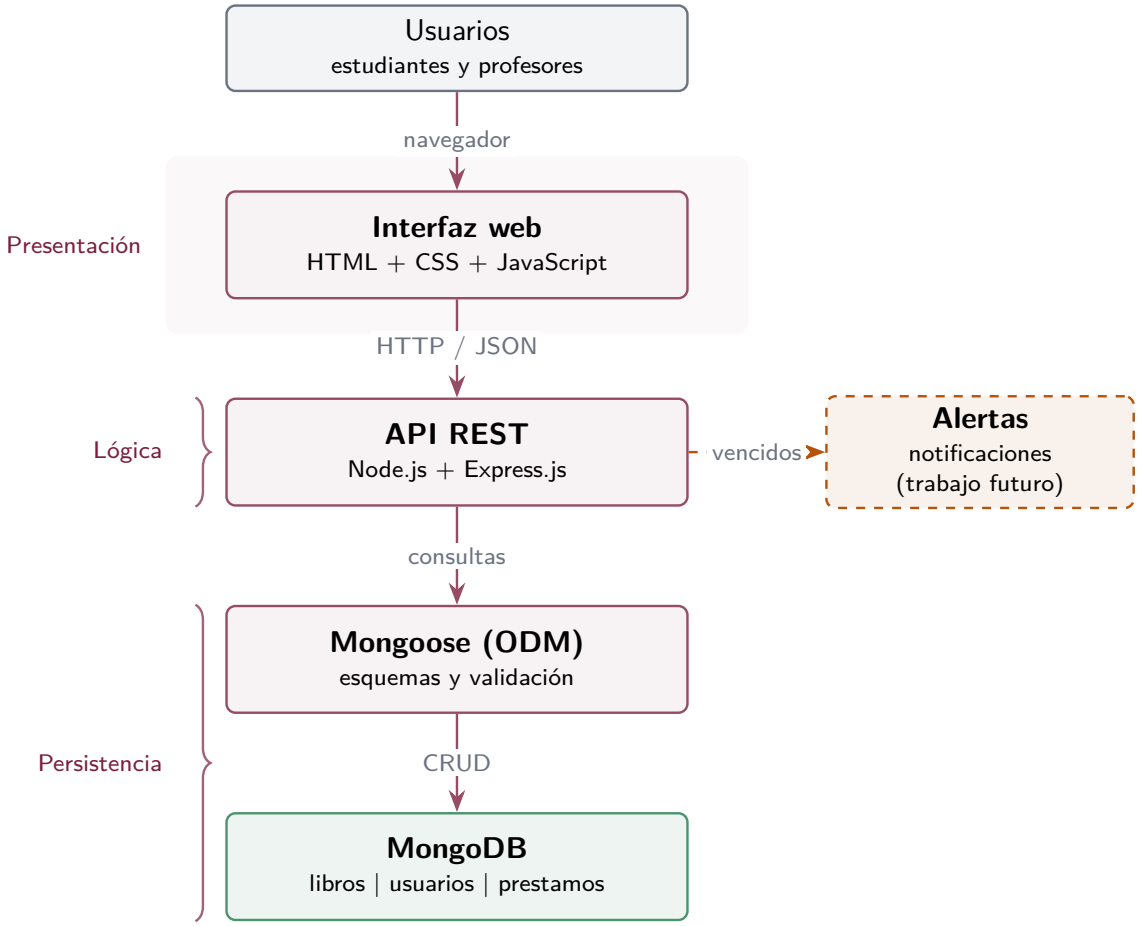


Figura 1: Arquitectura del sistema en capas. La línea punteada indica un componente previsto que aún no se implementa.

3.2 Flujo de un préstamo

El registro de un préstamo es la operación más delicada del sistema porque modifica dos colecciones y debe mantener consistente el inventario. La figura 2 muestra las validaciones que ejecuta el backend, en orden, y el código HTTP que devuelve en cada caso de rechazo.

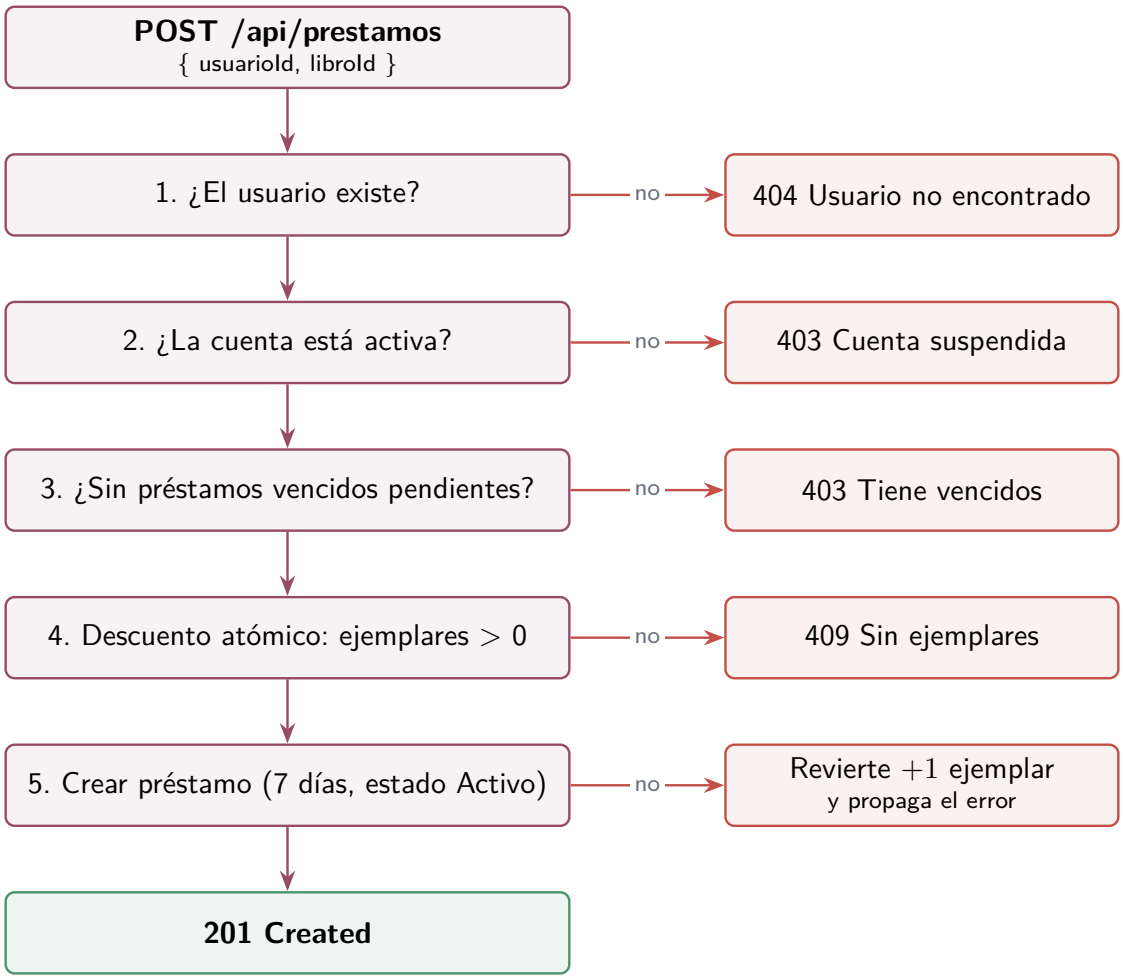


Figura 2: Validaciones del endpoint de préstamos y respuesta en cada caso.

4 Tecnologías utilizadas

Componente	Tecnología	Versión	Uso en el proyecto
Lenguaje base	JavaScript	ES2022	Lenguaje único para backend y frontend.
Entorno de ejecución	Node.js	26 (LTS)	Ejecución asíncrona del servidor.
Framework web	Express.js	4.x	Enrutamiento y gestión de la API REST.
Base de datos	MongoDB Community Server	9.0	Base de datos NoSQL orientada a documentos.
ODM	Mongoose	8.x	Esquemas, validación y consultas desde Node.js.
Cliente de base de datos	MongoDB Compass	1.53	Exploración visual, importación de JSON y consultas.
Interfaz	HTML, CSS y JavaScript	–	Estructura, diseño y lógica del cliente web.
Pruebas de API	Postman	–	Validación de los endpoints de la API.
Control de versiones	Git y GitHub	Git 2.54	Historial del código y entrega del proyecto.
Gestión del trabajo	Jira	–	Épicas e historias de usuario (proyecto BDDN).

Cuadro 1: Stack tecnológico del proyecto.

5 Origen de datos (MongoDB)

El uso de MongoDB como origen de datos principal aprovecha el almacenamiento de documentos JSON (BSON). El sistema trabaja con tres colecciones, cuya estructura se valida en la base de datos mediante `$jsonSchema` (el script completo se incluye en el anexo A).

5.1 Colecciones y campos

Colección `libros`. Cada libro es un documento autocontenido. Los autores se almacenan como un arreglo de cadenas dentro del mismo documento.

Campo	Tipo	Oblig.	Descripción y regla
`titulo`	string	Sí	Título del libro.
`autores`	array de string	Sí	Al menos un autor (`minItems: 1`); modelo embebido.
`isbn`	string	Sí	Identificador del libro; índice único.
`categoria`	string	Sí	Género o área (por ejemplo, Terror, Redes).
`edicion`	string	No	Edición (por ejemplo, `"3a"`).
`ejemplaresTotales`	number	No	Ejemplares que posee la biblioteca (mínimo 0).
`ejemplaresDisponibles`	number	Sí	Ejemplares que pueden prestarse (mínimo 0).

Cuadro 2: Estructura de la colección `libros`.

Colección `usuarios`. Perfiles de estudiantes y profesores.

Campo	Tipo	Oblig.	Descripción y regla
`nombre`	string	Sí	Nombre completo.
`correo`	string	Sí	Correo institucional; índice único.
`rol`	enum	Sí	`"estudiante"` o `"profesor"`.
`identificador`	string	No	Boleta o número de empleado.
`estadoCuenta`	enum	Sí	`"activa"` o `"suspendida"`.

Cuadro 3: Estructura de la colección `usuarios`.

Colección `prestamos`. Registra la relación entre usuario y libro mediante referencias.

Campo	Tipo	Oblig.	Descripción y regla
`usuario`	ObjectId	Sí	Referencia a `usuarios`.
`libro`	ObjectId	Sí	Referencia a `libros`.
`fechaPrestamo`	date	Sí	Fecha en que se registra el préstamo.
`fechaLimite`	date	Sí	Fecha límite de devolución (7 días).
`fechaDevolucion`	date o null	No	Se llena al devolver el libro.
`estado`	enum	Sí	`"Activo"`, `"Devuelto"` o `"Vencido"`.

Cuadro 4: Estructura de la colección `prestamos`.

5.2 Relaciones entre colecciones

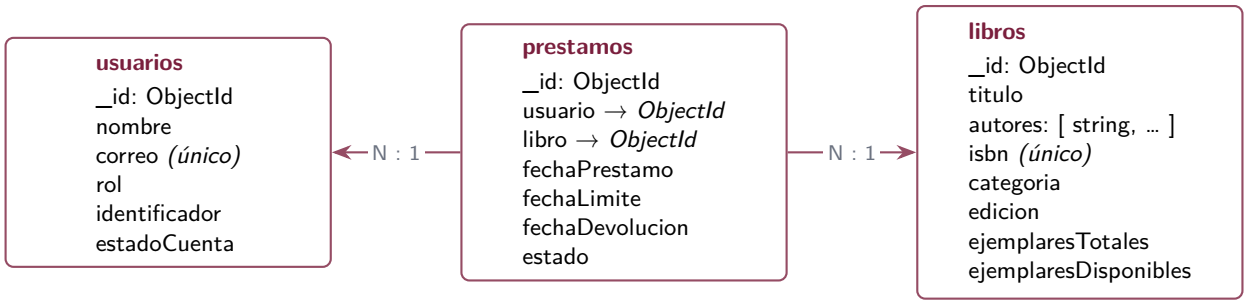


Figura 3: Modelo de datos: `prestamos` referencia a `usuarios` y `libros` por `ObjectId`; los autores van embebidos en `libros`.

5.3 Ejemplos de documentos

```
{
  titulo: "Drácula",
  autores: ["Bram Stoker"],
  isbn: "9789000030019",
  categoria: "Terror",
  edicion: "1a",
  ejemplaresTotales: 6,
  ejemplaresDisponibles: 6
}
```

Listado 1: Documento de la colección `libros`.

```
{
  usuario: ObjectId("..."),          // referencia a usuarios
  libro: ObjectId("..."),            // referencia a libros
  fechaPrestamo: ISODate("2026-11-02T18:30:00Z"),
  fechaLimite: ISODate("2026-11-09T18:30:00Z"),
  fechaDevolucion: null,
  estado: "Activo"
}
```

Listado 2: Documento de la colección `prestamos`.

5.4 Justificación del modelado NoSQL

- **Autores embebidos.** A diferencia de un modelo relacional, que requeriría una tabla intermedia libro–autor, el campo `autores` es un arreglo dentro del propio libro. Una sola lectura devuelve el libro completo, lo que optimiza la consulta del catálogo.
- **Referencias por `ObjectId` en préstamos.** Un préstamo se relaciona con un usuario y un libro que existen de forma independiente y cambian por su cuenta. Referenciar evita duplicar datos y permite recuperarlos con `$lookup` cuando se necesitan.
- **Contador de ejemplares dentro del libro.** `ejemplaresDisponibles` se guarda en el documento del libro para que la disponibilidad se consulte sin contar préstamos, y se actualiza con operadores atómicos (`$inc`).
- **Validación en dos niveles.** Los validadores `$jsonSchema` protegen a la base de datos aunque se escriba desde otra herramienta (por ejemplo, Compass), y los esquemas de Mongoose validan antes de llegar a ella.

5.5 Índices

Colección	Índice	Propósito
`libros`	`{ isbn: 1 }` único	Evitar duplicados y buscar por ISBN.
`libros`	texto en `titulo` y `autores`	Búsquedas bibliográficas de texto completo (español).
`libros`	`{ categoria: 1 }`	Filtrar el catálogo por género.
`usuarios`	`{ correo: 1 }` único	Evitar cuentas repetidas.
`usuarios`	`{ identificador: 1 }`	Buscar por boleta o número de empleado.
`prestamos`	`{ estado: 1, fechaLimite: 1 }`	Detectar préstamos vencidos eficientemente.
`prestamos`	`{ usuario: 1, estado: 1 }`	Préstamos activos de un usuario.
`prestamos`	`{ libro: 1 }`	Historial y reportes por libro.

Cuadro 5: Índices definidos para optimizar los tiempos de respuesta.

El índice de texto se crea con el script del modelo (anexo A). El buscador del catálogo utiliza, además, expresiones regulares insensibles a mayúsculas para admitir coincidencias parciales (por ejemplo, `"steph"` encuentra a Stephen King).

5.6 Datos de prueba

Para validar el sistema se cargaron **281 libros** (1 109 ejemplares) y 3 usuarios de ejemplo, organizados en archivos JSON por género dentro de la carpeta `data/` del repositorio.

Archivo	Libros	Contenido
`libros_tecnicos.json`	10	Bases de datos, programación, redes, sistemas operativos, IA, ingeniería de software y administración.
`libros_literatura_clasica.json`	5	Literatura clásica.
`libros_escolares.json`	51	Textos escolares y universitarios.
`libros_infantiles.json`	54	Literatura infantil y juvenil.
`libros_terror.json`	55	Terror y misterio.
`libros_romance.json`	53	Romance.
`libros_ciencia_ficcion.json`	53	Ciencia ficción.
Total	**281**	13 categorías.

Cuadro 6: Conjuntos de datos de prueba.

Sobre los ISBN

Los títulos y autores corresponden a obras reales. Los ISBN de los géneros agregados (escolares, infantiles, terror, romance y ciencia ficción) son valores generados para pruebas: únicos y con dígito verificador válido, pero no corresponden a las ediciones reales. Para un entorno de producción deben sustituirse por los ISBN de los ejemplares físicos.

6 Implementación del backend

6.1 Estructura del proyecto

```
biblioteca-web/
|-- data/            JSON de libros por genero (7 archivos, 281 libros)
|-- models/
|   `-- index.js     Esquemas Mongoose: Libro, Usuario, Prestamo
|-- routes/
|   |-- libros.js    Catalogo, categorias, alta y baja
|   |-- usuarios.js  Listado, busqueda y alta
|   `-- prestamos.js Prestamos, devoluciones y reporte
|-- public/          Interfaz web (index.html, styles.css, app.js)
|-- seed.js          Carga de datos e indices
|-- server.js        Servidor Express y conexion a MongoDB
`-- package.json
```

Listado 3: Estructura de carpetas del repositorio.

Método	Ruta	Descripción

6.2 API REST

Todas las respuestas son JSON. Los errores devuelven un objeto `{ error: "mensaje" }` con el código HTTP correspondiente.

Cuadro 7: Endpoints de la API REST.

Método	Ruta	Descripción
GET	`/api/libros`	Búsqueda con filtros `q`, `categoria`, `disponibles=1`, `pagina` y `limite` (máx. 100).
GET	`/api/libros/categorias`	Géneros con su total de libros (agregación `$group`).
POST	`/api/libros`	Alta de un libro.
DELETE	`/api/libros/:id`	Baja; se rechaza (409) si tiene préstamos activos.
GET	`/api/usuarios`	Listado; con `?q=` busca por nombre, boleta o correo (máx. 8).
POST	`/api/usuarios`	Alta de un usuario.
GET	`/api/prestamos`	Listado con libro y usuario (`populate`); filtro `?estado=`.
POST	`/api/prestamos`	Registra un préstamo (7 días).
POST	`/api/prestamos/:id/devolver`	Registra la devolución y repone el ejemplar.
GET	`/api/prestamos/reportes/mas-solicita`	Top 10 de libros más prestados.

6.3 Reglas de negocio

Descuento atómico de ejemplares. Para evitar que dos préstamos simultáneos dejen el inventario en negativo, la verificación y el descuento se hacen en una sola operación atómica: el filtro exige `ejemplaresDisponibles > 0` y el `$inc` se aplica solo si el documento cumple esa condición.

```
const libro = await Libro.findOneAndUpdate(
  { _id: libroId, ejemplaresDisponibles: { $gt: 0 } },
  { $inc: { ejemplaresDisponibles: -1 } },
  { new: true }
);
if (!libro) return res.status(409).json({ error: "No hay ejemplares disponibles." });
```

Listado 4: Descuento atómico de un ejemplar al registrar un préstamo.

Consistencia ante fallos. Si el ejemplar se descuenta pero falla la creación del préstamo, el backend repone el ejemplar y propaga el error. La devolución actúa de forma inversa: marca el préstamo como `Devuelto` (solo si estaba `Activo` o `Vencido`, lo que impide devolver dos veces) y suma un ejemplar.

Préstamos vencidos. Antes de listar préstamos o validar uno nuevo, el backend marca como `Vencido` todo préstamo `Activo` cuya `fechaLimite` ya pasó, apoyándose en el índice compuesto `{ estado, fechaLimite }`:

```
await Prestamo.updateMany(
  { estado: "Activo", fechaLimite: { $lt: new Date() } },
  { $set: { estado: "Vencido" } }
);
```

Listado 5: Actualización de préstamos vencidos.

Restricciones al prestar. No se presta a cuentas suspendidas ni a usuarios con préstamos vencidos pendientes de devolver (403), ni cuando no quedan ejemplares (409).

6.4 Reporte de libros más solicitados

El reporte usa una canalización de agregación que cuenta los préstamos por libro, ordena, limita a diez y recupera los datos del libro con `$lookup`:

```
Prestamo.aggregate([
  { $group:  { _id: "$libro", prestamos: { $sum: 1 } } },
  { $sort:   { prestamos: -1 } },
  { $limit:  10 },
  { $lookup: { from: "libros", localField: "_id",
               foreignField: "_id", as: "libro" } },
  { $unwind: "$libro" },
  { $project: { _id: 0, titulo: "$libro.titulo",
                categoria: "$libro.categoria", prestamos: 1 } }
])
```

Listado 6: Agregación del reporte de libros más solicitados.

6.5 Manejo de errores

Código HTTP	Causa
400	Datos inválidos: falla de validación de Mongoose, identificador mal formado o documento que no cumple el validador de MongoDB.
403	Cuenta suspendida o usuario con préstamos vencidos.
404	Usuario, libro o préstamo inexistente.
409	Valor duplicado (ISBN o correo), libro sin ejemplares o con préstamos activos.
500	Error interno del servidor.

Cuadro 8: Códigos de error devueltos por la API.

7 Interfaz web

La interfaz es una aplicación de una sola página servida por el propio Express. Se organiza en tres módulos accesibles desde la barra superior.

Catálogo. Muestra los libros en tarjetas con título, autores, categoría, ISBN y ejemplares disponibles (verde, ámbar o rojo según la existencia). Incluye buscador por título, autor o ISBN, filtro por categoría con el total de libros, opción de mostrar solo los disponibles, paginación y botones para prestar, agregar y eliminar libros.

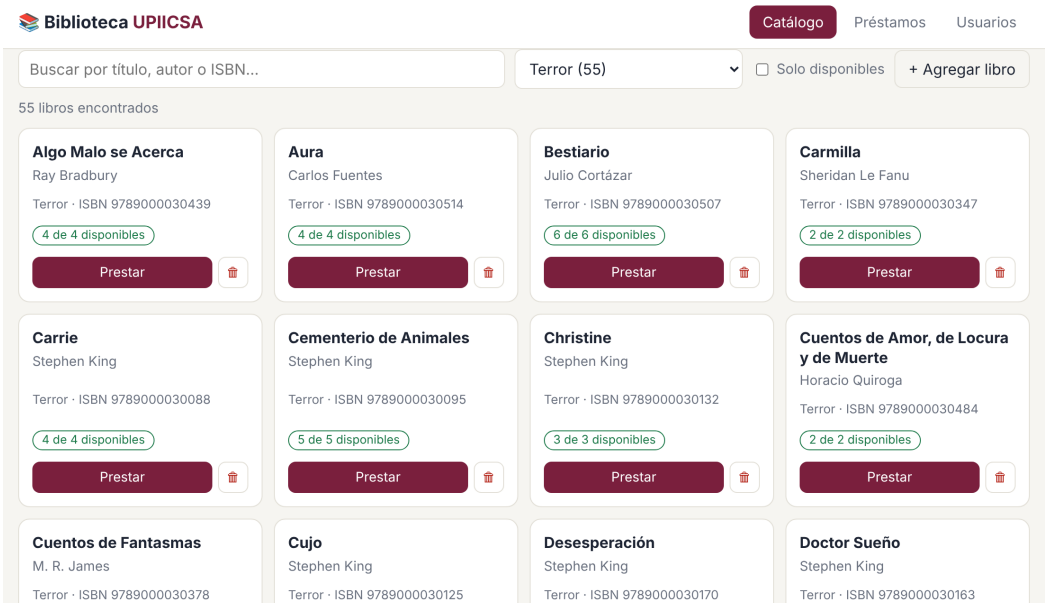


Figura 4: Módulo de catálogo filtrado por la categoría Terror (55 libros).

Préstamos. Al prestar un libro, el usuario se elige con un **buscador con resultados en vivo** por nombre, boleta o correo (desde 2 caracteres, máximo 8 coincidencias), en lugar de una lista completa, lo que escala a cualquier número de usuarios. Las cuentas suspendidas se señalan en los resultados.

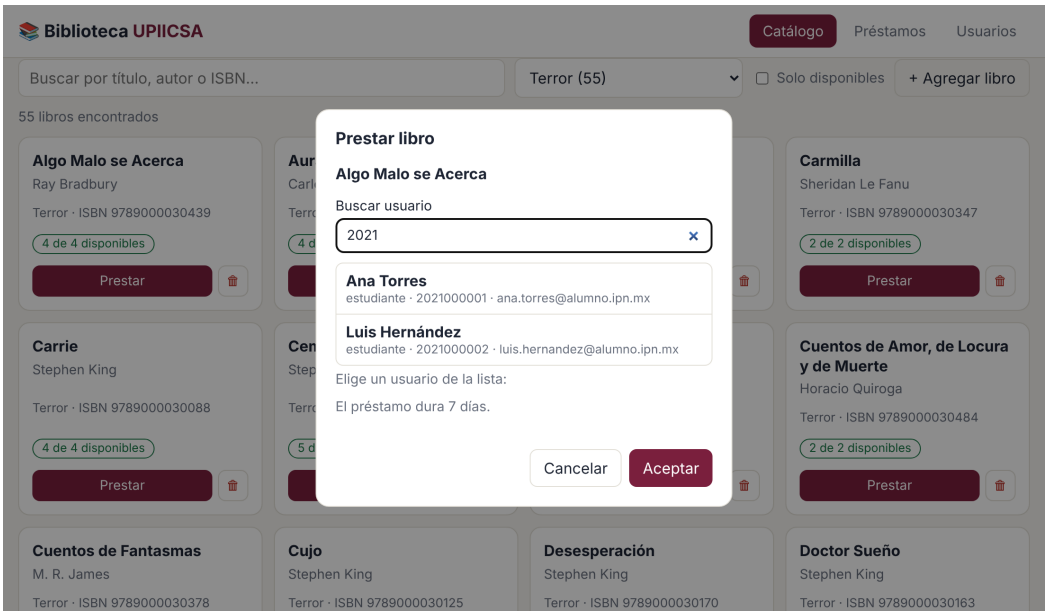


Figura 5: Búsqueda de un usuario por boleta al registrar un préstamo.

Usuarios. Lista a estudiantes y profesores con su rol, identificador y estado de cuenta, y permite registrar nuevos usuarios.



Figura 6: Módulo de usuarios.

La pestaña de préstamos lista cada préstamo con su estado (Activo, Vencido, Devuelto), permite filtrar por estado, registrar la devolución y consultar el reporte de libros más solicitados.

8 Instalación y ejecución

Las instrucciones corresponden a Windows con PowerShell.

8.1 Requisitos previos

1. **Node.js** (versión LTS) desde `https://nodejs.org`. Verificar con `node -v` y `npm -v`.
2. **MongoDB Community Server** desde `https://www.mongodb.com/try/download/community` (paquete `msi`). Se recomienda dejar marcada la opción *Install MongoDB as a Service*.
3. **MongoDB Compass** (opcional) para explorar los datos.
4. **Git** (opcional) para clonar el repositorio.

8.2 Pasos

```
git clone https://github.com/Raphael-27/biblioteca-nosql.git
cd biblioteca-nosql

npm install      # instala Express y Mongoose
npm run seed     # carga 281 libros, 3 usuarios e indices en la base "libreria"
npm start        # inicia el servidor en http://localhost:3000
```

Listado 7: Puesta en marcha del proyecto.

Después se abre `http://localhost:3000` en el navegador. El comando `npm run seed` puede ejecutarse varias veces sin duplicar libros porque usa el ISBN como llave; con `npm run seed -- --reset` se vacían las colecciones antes de cargar.

Configuración. La aplicación se conecta por defecto a `mongodb://127.0.0.1:27017/libreria`. Para usar otra base o servidor se define la variable de entorno `MONGO_URI` (por ejemplo, `$env:MONGO_URI="mongodb://127.0.0.1:27017/biblioteca"` en PowerShell). Si se usa el script del anexo A, que crea la base `biblioteca`, debe apuntarse la aplicación a ese nombre.

8.3 Problemas frecuentes

Cuadro 9: Problemas frecuentes durante la instalación.

Síntoma	Solución
`ECONNREFUSED 127.0.0.1:27017` al ejecutar `seed` o `start`	MongoDB no está encendido. Iniciar el servicio *MongoDB Server* en Servicios de Windows.
No existe el servicio MongoDB	Crear la carpeta de datos y arrancar el servidor a mano: `mkdir C:\data\db` y ejecutar `mongod.exe` desde `C:\Program Files\MongoDB\Server\9.0\bin`. La ventana debe permanecer abierta.
`npm.ps1 no se puede cargar porque la ejecución de scripts está deshabilitada`	Ejecutar una vez `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` y reabrir PowerShell.
Error de ISBN o correo duplicado (409)	Ya existe un registro con ese valor; usar otro o ejecutar `npm run seed -- --reset`.

9 Pruebas

9.1 Pruebas funcionales

El equipo verificó manualmente los flujos principales de la interfaz sobre una instancia local de MongoDB 9.0: búsqueda por texto, filtro por categoría, registro de un préstamo, devolución, y alta de libros y usuarios.

9.2 Plan de pruebas de la API

El cuadro siguiente define los casos que se validan con Postman sobre los endpoints (cuadro 7), incluyendo los de rechazo, que comprueban las reglas de negocio.

Cuadro 10: Plan de pruebas de la API REST.

Id	Endpoint	Caso	Resultado esperado
P01	GET `/api/libros`	Consulta sin filtros	200; 24 libros y total 281.
P02	GET `/api/libros?q=stephen`	Búsqueda parcial por autor	200; solo libros con coincidencia.
P03	GET `/api/libros?categoria=Terror`	Filtro por categoría	200; total 55.
P04	GET `/api/libros/categorias`	Resumen por género	200; 13 categorías.
P05	POST `/api/libros`	Libro válido	201; documento creado.
P06	POST `/api/libros`	ISBN repetido	409.
P07	POST `/api/libros`	Falta el título	400.
P08	POST `/api/usuarios`	Usuario válido, correo repetido y rol inválido	201, 409 y 400.
P09	POST `/api/prestamos`	Préstamo válido	201; el libro baja un ejemplar.
P10	POST `/api/prestamos`	Libro sin ejemplares	409.
P11	POST `/api/prestamos`	Usuario con cuenta suspendida	403.
P12	POST `/api/prestamos`	Usuario con préstamo vencido	403.
P13	POST `/api/prestamos/:id/devolver`	Devolución de un préstamo activo	200; el libro sube un ejemplar.
P14	POST `/api/prestamos/:id/devolver`	Devolver un préstamo ya devuelto	404.
P15	DELETE `/api/libros/:id`	Libro con préstamos activos	409.
P16	GET `/api/prestamos/reportes/mas-`	Reporte	200; máximo 10 libros ordenados.

10 Gestión del proyecto

10.1 Planeación en Jira

El trabajo se organizó en el proyecto **BDDN** de Jira, con siete épicas y dieciocho historias de usuario.

Épica	Alcance	Historias
BDDN-1	Modelo de base de datos (colecciones, validadores, índices, datos iniciales)	3
BDDN-2	API REST con Node.js y Express (CRUD, Mongoose, préstamos)	4
BDDN-3	Búsqueda y disponibilidad en tiempo real	2
BDDN-4	Interfaz web (catálogo, usuarios, préstamos)	3
BDDN-5	Alertas de préstamos vencidos	2
BDDN-6	Reportes analíticos	2
BDDN-7	Pruebas y documentación	2

Cuadro 11: Épicas del proyecto.

10.2 Control de versiones

El código fuente se versiona con Git y se publica en GitHub: `https://github.com/Raphael-27/biblioteca-nosql`. El archivo `.gitignore` excluye `node_modules`, por lo que el repositorio contiene solo el código, los datos de prueba y la documentación; las dependencias se reinstalan con `npm install`.

11 Estado de cumplimiento, limitaciones y trabajo futuro

11.1 Cumplimiento de los objetivos

Cuadro 12: Grado de cumplimiento de los objetivos específicos.

Obj.	Estado	Evidencia o pendiente
O1	**Parcial**	Alta, consulta y baja de libros. Falta la modificación (actualización) desde la API y la interfaz.
O2	**Parcial**	Alta, listado y búsqueda de usuarios. Faltan edición y baja.
O3	**Cumplido**	Préstamo y devolución con operaciones atómicas y reposición ante fallos. No se usan transacciones multidocumento.
O4	**Cumplido**	`ejemplaresDisponibles` se actualiza de forma atómica y se consulta en cada carga del catálogo.

Obj.	Estado	Evidencia o pendiente
O5	**Parcial**	Detección automática de vencidos y bloqueo de nuevos préstamos. Falta el envío de notificaciones.
O6	**Parcial**	Depende de O1 y O2 (altas y bajas disponibles; modificaciones pendientes).
O7	**Cumplido**	Búsqueda por título, autor, ISBN y categoría, con paginación.
O8	**Parcial**	Reporte de libros más solicitados. Los usuarios con retrasos se consultan filtrando préstamos por estado Vencido; falta un reporte agregado propio.
O9	**Cumplido**	Índices únicos, de texto, simples y compuestos definidos en el modelo y en el script de carga.
O10	**Cumplido**	Diez endpoints REST con manejo uniforme de errores (cuadro 7).

11.2 Limitaciones

- No hay autenticación: cualquier persona con acceso a la aplicación puede operar todos los módulos.
- Los ISBN de los géneros agregados son de prueba (ver la sección 5.6).
- La disponibilidad se refresca al consultar el catálogo; no hay actualización automática en pantalla.
- La detección de vencidos se ejecuta al consultar o registrar préstamos, no mediante un proceso programado.

11.3 Trabajo futuro

1. Completar el CRUD: edición de libros y usuarios, y baja de usuarios.
2. Reporte agregado de usuarios con retrasos, con `$lookup` entre `prestamos` y `usuarios`.
3. Notificaciones automáticas por correo para préstamos próximos a vencer y vencidos, con una tarea programada.
4. Autenticación con roles (administrador y bibliotecario).
5. Multas, renovaciones y reservaciones.
6. Colección de Postman y pruebas automatizadas incluidas en el repositorio.

12 Conclusiones

El proyecto permitió aplicar los conceptos de bases de datos NoSQL en un caso realista. El modelo documental de MongoDB simplificó la representación de los libros: con los autores embebidos, una sola lectura devuelve el registro completo, mientras que las referencias por `ObjectId` mantienen independientes a usuarios y libros en los préstamos. La combinación de validadores `$jsonSchema`, esquemas de Mongoose e índices (únicos, de texto y compuestos) dio integridad y buen desempeño en las consultas.

En el control de inventario, el uso de operaciones atómicas (`findOneAndUpdate` con `$inc`) resolvió el riesgo de inconsistencias cuando varios usuarios piden el mismo libro, sin necesidad de transacciones más costosas. La API REST separó con claridad la lógica de negocio de la interfaz, y la arquitectura por capas deja preparado el camino para incorporar notificaciones, autenticación y reportes adicionales.

Como siguiente paso, el equipo identificó con precisión los pendientes (sección 11), principalmente la edición de registros, el reporte de usuarios con retrasos y las notificaciones automáticas.

Referencias

[1] MongoDB, Inc. *MongoDB Manual*. https://www.mongodb.com/docs/manual/

[2] MongoDB, Inc. *MongoDB Compass*. https://www.mongodb.com/docs/compass/

[3] Mongoose. *Mongoose Documentation*. https://mongoosejs.com/docs/

[4] Express.js. *Express Documentation*. https://expressjs.com/

[5] OpenJS Foundation. *Node.js Documentation*. https://nodejs.org/docs/latest/api/

[6] Postman. *Postman Learning Center*. https://learning.postman.com/

A Script del modelo de datos (mongosh)

Script que crea las colecciones con sus validadores e índices. Se ejecuta con `mongosh "mongodb://127.0.0.1:27017" modelo_de_datos_biblioteca.js`. Crea la base `biblioteca`; si se desea usar la base `libreria`, basta cambiar el nombre en la primera línea.

```
const bd = db.getSiblingDB("biblioteca");

["libros", "usuarios", "prestamos"].forEach(c => {
  if (bd.getCollectionNames().includes(c)) bd.getCollection(c).drop();
});

/* 2. COLECCION: libros */
```

```
bd.createCollection("libros", {
  validator: { $jsonSchema: {
    bsonType: "object",
    required: ["titulo", "autores", "isbn", "categoria", "ejemplaresDisponibles"],
    properties: {
      titulo:    { bsonType: "string" },
      autores:   { bsonType: "array", minItems: 1, items: { bsonType: "string" } },
      isbn:      { bsonType: "string" },
      categoria: { bsonType: "string" },
      edicion:   { bsonType: "string" },
      ejemplaresTotales:     { bsonType: "number", minimum: 0 },
      ejemplaresDisponibles: { bsonType: "number", minimum: 0 }
    }
  }}
});

/* 3. COLECCION: usuarios */
bd.createCollection("usuarios", {
  validator: { $jsonSchema: {
    bsonType: "object",
    required: ["nombre", "correo", "rol", "estadoCuenta"],
    properties: {
      nombre:        { bsonType: "string" },
      correo:        { bsonType: "string" },
      rol:           { enum: ["estudiante", "profesor"] },
      identificador: { bsonType: "string" },
      estadoCuenta:  { enum: ["activa", "suspendida"] }
    }
  }}
});

/* 4. COLECCION: prestamos */
bd.createCollection("prestamos", {
  validator: { $jsonSchema: {
    bsonType: "object",
    required: ["usuario", "libro", "fechaPrestamo", "fechaLimite", "estado"],
    properties: {
      usuario:         { bsonType: "objectId" },
      libro:           { bsonType: "objectId" },
      fechaPrestamo:   { bsonType: "date" },
      fechaLimite:     { bsonType: "date" },
      fechaDevolucion: { bsonType: ["date", "null"] },
      estado:          { enum: ["Activo", "Devuelto", "Vencido"] }
    }
  }}
});

/* 6. INDICES (optimizacion de tiempos de respuesta) */
bd.libros.createIndex({ isbn: 1 }, { unique: true });
bd.libros.createIndex({ titulo: "text", autores: "text" },
                      { default_language: "spanish", name: "busqueda_texto" });
bd.libros.createIndex({ categoria: 1 });
bd.usuarios.createIndex({ correo: 1 }, { unique: true });
bd.usuarios.createIndex({ identificador: 1 });
bd.prestamos.createIndex({ estado: 1, fechaLimite: 1 });
bd.prestamos.createIndex({ usuario: 1, estado: 1 });
```

```
bd.prestamos.createIndex({ libro: 1 });

print("Modelo de datos creado exitosamente.");
```